

Notes on Chapter 10: Classical Planning

chapters 8 and 9 of the textbook are an in-depth discussion of first-order logic

while the use of first-order logic is important and useful in AI, it is quite technical, and since this is a survey course we will skip to be able to spend more time on other topics

- it is worth noting that the “logical approach” to AI is one of the major streams of AI research, and many AI researchers essentially study logic
- this logical approach does not yet seem to have hit the “big time” in the way that machine learning and neural nets have, i.e. there are, as of yet, not many practical applications of logic-oriented AI that
 - SAT solvers may be the biggest practical success so far
- so if you are interested in more of the logical approach to AI, we encourage you to start by reading chapters 8 and 9 of the textbook for your own interest

Planning

planning is one of the classic AI problems

it has been used as the basis for applications like controlling robots and having conversations

a plan is a sequence of actions

an action is a transformation of a state, so a plan can be thought of as a series of transformations of some initial state

we usually specify an initial state and a goal state, and then try to find (a short!) plan that transform the initial state into the goal state

- in more realistic planning problems, we might not know exactly what initial state we are in!

this states and actions model of planning is quite general, and so can be applied to many different kinds of problems

in general, planning is an extremely difficult problem, and planners are usually out-performed by solvers customized specifically for a particular problem

Example Planning Problem: Making Tea

suppose you have a robot that can serve tea

think about what the robot must do to make tea, e.g.:

- it must put water in the kettle
- heat the kettle
- get a cup
- pour hot water into the cup (after the water is hot enough)
- get a tea bag
- leave the tea bag in the water for enough time
- remove the tea bag
- add milk
- add sugar
- mix the tea
- serve the tea

there are many, many actions to consider!

and most actions consist of many smaller sub-actions

and in some situations very long sub-plans might be triggered

- e.g. if there is no milk, the robot might go to the store, or ask if creme would be an acceptable substitute
- e.g. if the robot drops the spoon on the floor, then it must pick it up and clean it before continuing

time and sensing is important as well

- how long should the robot mix the tea for?

Example Planning Problem: Getting Dressed

suppose you want to put your socks and shoes on

suppose you have:

- a left sock
- a right sock
- a left shoe
- a right shoe

one possible plan for putting these on is:

- put left sock on left foot
- put right sock on right foot
- put left shoe on left foot
- put right shoe on right foot

another possible plan is:

- put left sock on left foot
- put left shoe on left foot
- put right sock on right foot
- put right shoe on right foot

for this planning problem, what matters is that the left sock be put on before the left shoe, and the right sock be put on before the right shoe

- for many actions, the exact order doesn't matter
- plus, it might be possible to do some actions in parallel, e.g. maybe you could somehow hop into both shoes at the same time

note that we made the somewhat unusual choice that there are "left socks" and "right socks"

- more realistically, we might say we have different socks, sock1 and sock2
- it doesn't matter on which foot sock1 and sock2 go, so even more plans are possible
 - i.e. you could swap sock1 and sock2 in any correct plan to get another correct plan

Example Planning Problem: Blocks World

suppose you have three cubical blocks, A, B, and C, and a robot arm that can pick up and move one block at a time

suppose they are initially arranged on a table like this:

C	
B A	initial state

A and B are on the table, and C is on top of A

suppose that we want to re-arrange them into this state:

A	
B	goal state
C	

furthermore, suppose we re-arrange blocks by picking up one block at a time and moving it to the table, or on top of another block

- we can only pick up a block if there is nothing on top of it

it's easy to see that these actions will solve this particular problem:

- move C to the table
- move B on top of C
- move A on top of B

Dialog

a conversation between two people can be thought of as series of actions

- a conversational action is sometimes called a **speech act**

for example, consider this little conversation:

Customer: Can you tell me where I can find bread?

Employee: It's in aisle 2.

Customer: Thanks!

here, the customer has the goal of "finding bread"

there are various actions they could do to achieve this goal:

- walk around the store searching for bread
- finding a store map that lists where bread is
- ask someone for help

in the conversation, the person has decided to do the "ask for help" action

- note that for this action to succeed, a few things must be true as a pre-condition, e.g.:
 - they must be near an employee and speak loudly and clearly enough in English
 - they assume that the employee knows where the bread is, and they assume that the employee recognizes this and so realizes the question is in fact a request for the location of the bread — not a request to be told whether or not the employee knows where the bread is
 - i.e. the customer does not expect the employee to answer "Yes, I do know where you can find it?"
- so asking for help is, more specifically, a request for the location of the bread
- the employee responds with an assertion, i.e. they assert the location of the bread

- the customer then says “thanks”, which indicates to the employee that they appreciate the help, are satisfied with their answer, and that the dialog is over
- so we could say the customer performed a 2-action plan:
 - action 1: requesting the location of the bread
 - action 2: saying thank you

if we wanted to create an agent that could participate in conversations, then we could think of it as a sort of planning problem with speech actions

Blocks World in More Detail

suppose you have three cubical blocks, A, B, and C, and a robot arm that can pick up and move one block at a time

suppose they are initially arranged on a table like this:

```

      C
    B A   initial state
-----

```

A and B are on the table, and C is on top of A

suppose that we want to re-arrange them into this state:

```

A
B   goal state
C
---
```

to be able to write programs to solve planning problems, we need a precise representation of states, and also of actions that can be applied to states

logic is often user for states, e.g. first guess you could represent a state like this:

- Block(x) means object x is a block
- On(x, y) means object x is on top of object y
- Clear(x) means there is nothing on top of object x

then we could model the initial state like this:

```

      C
    B A   initial state
-----

On(B, Table) & On(A, Table) & On(C, A)
& Clear(B) & Clear(C)
& Block(A) & Block(B) & Block(C)

```

clear(x) seems a bit unusual, but is used because most planning languages (such as the one used in the textbook) don't support quantifiers in state descriptions

- i.e. if we had quantifiers we could say there is nothing on top of B with the sentence (for all blocks x).!on(x,B)
- but our planning language doesn't have quantifiers, so we instead use clear(B) to explicitly indicate that B can be picked up

now lets consider actions

an action modifies a state, so we need to know at least two things for every action:

- whether or not the action can be applied to a given state; this is called the actions **pre-condition**
- how the state changes after the action is applied; this is called the action's **effect**

for example:

```
Move(b, x, y)
  Pre-condition: On(b,x) & Clear(b) & Clear(y)
  Effect: On(b,y) & Clear(x) & !On(b,x) & !Clear(y)
```

this is an **action schema**, i.e. a template that describes all possible move actions

- it is a scheme because b, x, and y are variables, and need to be filled in by actual block names
- for instance, in a blocks world with 3 blocks, there are, at most, $3*2*1=6$ move actions involving just blocks:

```
Move(A, B, C)
Move(A, C, B)
Move(B, A, C)
Move(B, C, A)
Move(C, A, B)
Move(C, B, A)
```

Move(b, x, y) is the action that moves block b from x to y, and it has this pre-condition:

- b must clear so that b can be picked up
- y must be clear so that b can be placed on top of it
- b must be on top of x

the effect of Move(b, x, y) is:

- b is on top of y
- y is no longer clear (because b has been put on it)
- x is clear (because b has been moved from on top of it)
- b is no longer on top of x

the table presents a bit of a problem

- Move(B, Table, C) has the effect Clear(Table); so we will take Clear(Table) to mean that a block can be placed on the table
- Move(C, A, Table) has the effect !Clear(Table); but the table should always have space for a block, so that cannot happen

the usual solution to this is to add a second action schema:

```
MoveToTable(b, x)
  Pre-condition: On(b, x) & Clear(b)
  Effect: On(b, Table) & Clear(x) & !On(b, x)
```

again this is an action scheme: filling in b and x with block names from our 3-block world results in $3*2=6$ actions

blocks world is an interesting problem because it is relatively simple, and general-purpose planners without heuristics specific to blocks world can have trouble finding short plans that solve particular problems

Propositional Planning

in practice, many planners use *propositions* instead of full-blown predicate logic to describe actions and states

in propositional planning, states can be thought of as variables and-ed together, with the understanding that a variable that does not appear in a state is false

- i.e. states list only true facts; any facts not listed are assumed to be false

action schemas can then be simplified like this:

```
SomeAction
  Pre-condition: a,b,c
    Add list: e,f
    Delete list: a,b
```

the idea is that SomeAction can be applied to any state that has {a,b,c} as a subset

e.g. suppose the state $s=\{a,b,c,d\}$; then applying SomeAction to s gives the state $\{c,d,e,f\}$

and when it is applied, it does two things: it adds {e,f}, and removes {a,b}

in general, we have this equation for applying an action a to some state s :

$$\text{Result}(a, s) = (s - \text{Del}(a) \cup \text{Add}(a))$$

$\text{Del}(a)$ is the delete list for action a , and $\text{Add}(a)$ is the add list for a

this shows us how to go *forward* from one state to another by applying an action

Planners: Forward Planners

a **forward planner**, or **progression planner** starts at the *initial* state, and applies actions in an effort to find a path to the goal state

until about the year 2000, forward planning was thought to be too inefficient to be practical

some apparent problems with forward planning are:

- how can you choose what action to apply to a state when there are, say, thousands of actions to choose from?
 - e.g. the textbook gives the example of buying a book online using the action scheme $\text{Buy}(\text{isbn})$, where isbn is an ISBN number for a book
 - an ISBN has at least 10 digits, so a planner would have to essentially enumerate all 10 billion ISBN numbers to choose among $\text{Buy}(\text{isbn})$ actions
- another problem with forward planning is that the search spaces can be huge for even problems that don't seem that big
 - e.g. the textbook gives the problem 10 airports, where each airport has 5 planes and 20 pieces of cargo
 - the problem is to move all the cargo at airport A to airport B
 - the solution is easy: put all 20 pieces of cargo into a plane at A, and fly it to B, and unload all the cargo (for a total of 41 actions)
 - the difficulty is that the branching factor is very large, i.e. there are 50 planes in total that can fly to 9 different airports, and 200 pieces of cargo that can be loaded or unloaded
 - between 450 and 10450 actions per state!
 - if we assume 2000 actions per state on average, then the obvious 41 move solution is in a search tree with 2000^{41} nodes

surprisingly, forward planning's reputation changed in the early 2000s as good heuristics were found that lead to the creation of efficient forward planning systems

Planners: Backward Planners

a **backward planner**, or **regression planner**, starts at the *goal* state, and works backwards from that to try to find a path to the state

for many decades, backwards planners were thought to be inherently more efficient than forward planners because they only consider actions that are relevant to the goal

the reason for their presumed superiority was they result in smaller branching factors because they focus on relevant states

- when you start at the goal state, you can focus your search just on the actions that are relevant to achieving that state

however, it does require that you apply actions "in reverse", which results in having to deal with *sets of states* instead of individual states (as in forward planning)

Planning Heuristics

it appears now that the best planners are forward planners

the essential reason is that very good general-purpose, and problem-specific, heuristics have been found for forward planners, but similarly good heuristics have not been found for backwards planners

for example, one good general-purpose forward planning heuristic is to ignore pre-conditions of actions: this gives a relaxed version of the planning problem that is still hard to solve, but for which pretty good approximation algorithms exist

another useful technique is to create a so-called **planning graph**, which can give good estimates of how many actions might be needed to reach a goal

- if you are curious, see the textbook for details on how to create planning graphs, and how a planning graph can be used to make a complete planner known as GraphPlan

Planners: SATPlan

another efficient approach to basic planning is to convert a planning problem into a SAT problem, and then to use a SAT solver to find a solution

the trick here is to come up with a propositional encoding of a SAT problem

- if you are curious, section 10.4.1 of the textbook outlines how to do this conversion: it has many steps, but it is fairly straightforward

Planners: Partial Order Planners

before the popularity of techniques like forward planning with good heuristics, GraphPlan, and SATPlan, many AI researchers thought partial order planners was the most promising approach to planning

partial order planners don't force all actions to be executed in a particular order: for some actions the order might not matter

a partial order plan is a set of actions plus a set of constraints on those actions of the form *Before(a_i, a_j)* (which means action a_i occurs before action a_j)

partial order planners search through **plan space**, instead of state space

a partial-order planners starts with the **empty plan**, and then tries inserting actions in a way that corrects some **flaw** in the current plan

- a flaw is anything keeps the plan from being true
- the order in which flaws are addressed are important, and so backtracking might be necessary

partial-order planners have not seen the same performance improvements as total-order state-based planners

- it seems that this is largely due to the difficulty of finding good heuristics for them

And More ...

planning is still an active area of research in AI, and new planners and techniques are being proposed all the time

also, here we only discussed the simplest sort of planning problems

real-life planning problems introduce other complexities, such as:

- resource constraints, e.g.
 - the plan you create might not be allowed to over-use some action, e.g. perhaps a robot arm can only be moved three times in a row before it over-heats and must rest
 - you might have to figure out a plan in a limited amount of time, i.e. the planner itself has a time constraint on how long it can run for
- domains might be non-deterministic, i.e. some actions might have random effects and so you can't be sure what state you are in after they are applied
- multi-agent planning occurs when you have more than one agent that can participate in the plan; the agents must coordinate their actions to solve problems
 - note that multi-agent planning could involve just two agents, e.g. in dialog, or in hundreds (or more) agents working together (like a swarm of insects or flock of birds)

finally, we note one other interesting problem related to planning: **plan recognition**

imagine you are watching an agent perform some actions: can you determine from the actions alone what plan, if any, the agent is following?

if you can figure out what plan an agent is following, then you might be able to predict what they are going to do (and then help them out)

- e.g. imagine your Google Home watches you cooking in the kitchen, and from your actions comes to believe that you are trying to cook spaghetti; it might suggest a good recipe, or tell you that you are missing certain ingredients, etc.